

CSCE 46103/56103 Artificial Intelligence – Fall 2026

**State-Space Search: BFS, DFS,
and Uniform-Cost Search**

Jungtaek Kim

`jungtaek.kim@uark.edu`

University of Arkansas

August 25, 2026

Undergraduate Research Opportunity

Professor Qinghua Li is seeking undergraduate research assistants

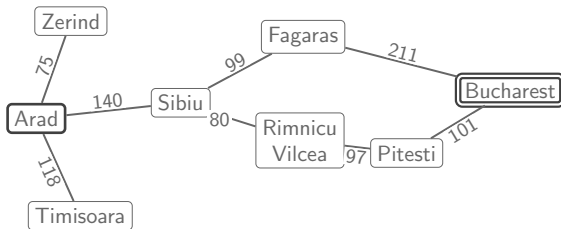
A research project is recruiting students with some background in artificial intelligence or machine learning.

- ▶ **Interested students:** contact Professor Li directly at qinghual@uark.edu.
- ▶ **Learn more:** haic.uark.edu

Today's Topics

1. **State-Space Search:** search nodes, the frontier, reached states, and duplicate handling
2. **Breadth-First Search:** shallowest-first exploration with a FIFO frontier
3. **Depth-First Search:** deepest-first exploration with a LIFO frontier
4. **Uniform-Cost Search:** lowest- $g(n)$ exploration with a priority frontier
5. **Strategy Comparison:** completeness, cost-optimality, time, and space

Route Choice



Fewer roads

Arad → Sibiu → Fagaras → Bucharest

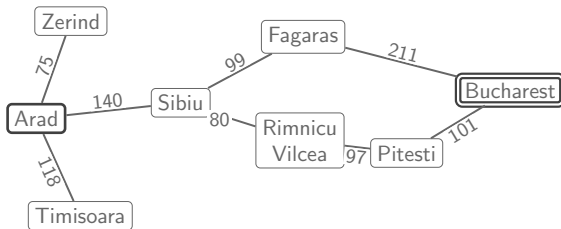
depth = 3, $g = 450$

Lower distance

Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

depth = 4, $g = 418$

Route Choice



Fewer roads

Arad → Sibiu → Fagaras → Bucharest

depth = 3, $g = 450$

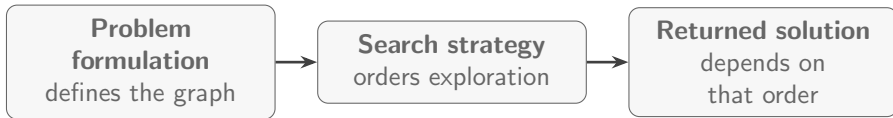
Lower distance

Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

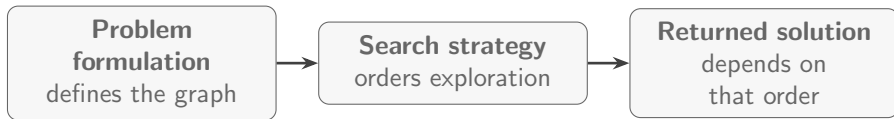
depth = 4, $g = 418$

**When several partial routes are waiting,
which one should the agent explore next?**

Formulation and Exploration



Formulation and Exploration



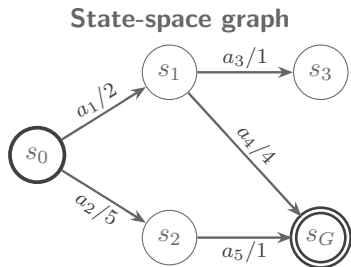
Shallowest first
Breadth-first search
FIFO frontier

Deepest first
Depth-first search
LIFO frontier

Lowest g first
Uniform-cost search
Priority frontier

Same formulation; different exploration order.

Search Nodes



Search node

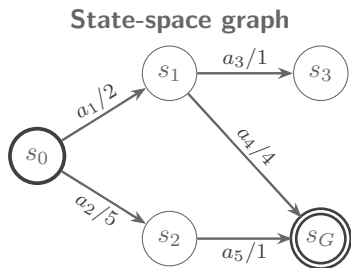
state: current abstract state

parent: preceding search node

action: transition from parent

$g(n)$ and **depth:**
path summaries

Search Nodes



Search node

state: current abstract state

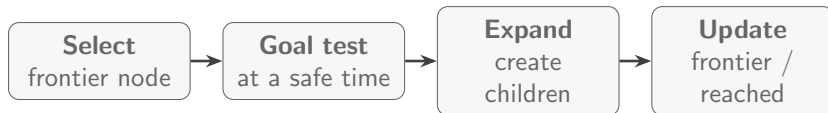
parent: preceding search node

action: transition from parent

$g(n)$ and **depth:**
path summaries

One graph vertex represents a state. Different paths can create different search nodes for that state; parent links recover the returned solution.

Frontier and Reached



Frontier

Search nodes generated but not yet selected. Its ordering rule *is* the search strategy.

Reached

States already generated, including states still on the frontier. It prevents loops and controls duplicate paths.

Duplicate Policies

Strategy	Frontier discipline	Reached information
BFS	FIFO queue	reached / not reached
DFS	LIFO stack	reached / not reached
UCS	minimum- g priority queue	best g and parent

Duplicate Policies

Strategy	Frontier discipline	Reached information
BFS	FIFO queue	reached / not reached
DFS	LIFO stack	reached / not reached
UCS	minimum- g priority queue	best g and parent

BFS and DFS

The first generated path claims the state. Later duplicate paths are discarded.

Uniform-cost search

A cheaper path replaces the frontier node and its parent. A costlier or equal path is discarded.

Duplicate handling is part of the algorithm, not a cleanup step.

Search Criteria

Complete

Will it find a solution when one exists?

Cost-optimal

Will it return a minimum-cost solution?

Time

How many nodes or actions are considered?

Space

How many nodes must remain in memory?

Search Criteria

Complete

Will it find a solution when one exists?

Cost-optimal

Will it return a minimum-cost solution?

Time

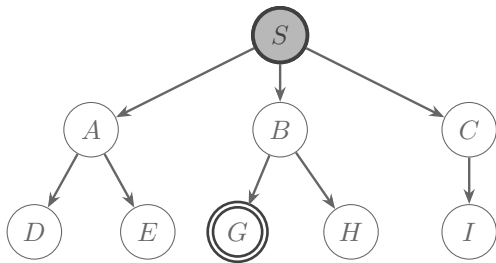
How many nodes or actions are considered?

Space

How many nodes must remain in memory?

b	branching factor	d	shallowest-solution depth
m	maximum search-tree depth	C^*	optimal solution cost
ϵ	positive lower bound on action cost	$g(n)$	cost from the start to n

Breadth-First Ordering

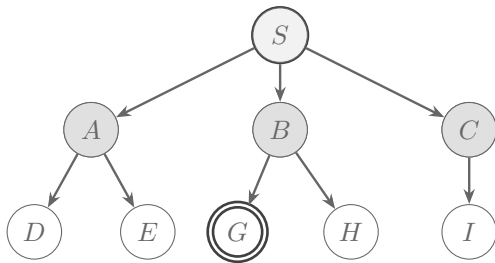


Rule

Select the shallowest node. A FIFO queue preserves generation order within a depth.

Queue: S

Breadth-First Ordering



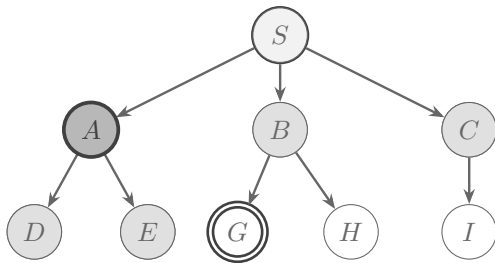
Rule

Select the shallowest node. A FIFO queue preserves generation order within a depth.

After expanding S :

A B C

Breadth-First Ordering



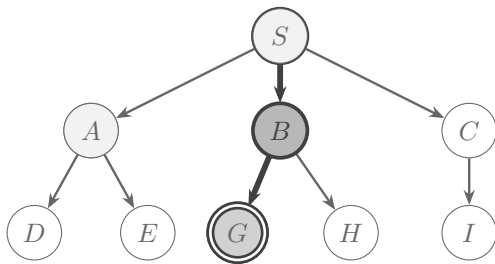
Rule

Select the shallowest node. A FIFO queue preserves generation order within a depth.

After expanding A :



Breadth-First Ordering



Rule

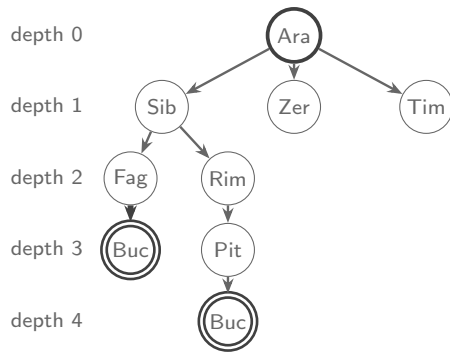
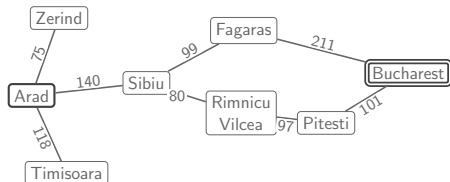
Select the shallowest node. A FIFO queue preserves generation order within a depth.

While expanding B :

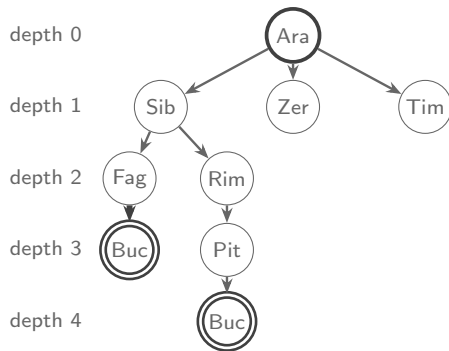
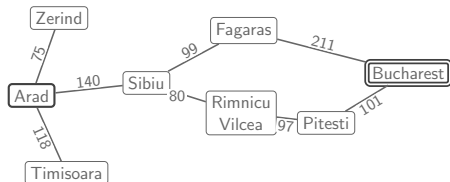
G is generated at depth 2.

Early goal test: return.

Romania by Levels



Romania by Levels



Returned route	Actions	Distance
Arad–Sibiu–Fagaras–Bucharest	3	450
Arad–Sibiu–Rimnicu–Pitesti–Bucharest	4	418

BFS minimizes the number of actions, not arbitrary path cost.

Breadth-First Tradeoffs

b : maximum branching factor d : shallowest solution depth

Property	Breadth-first search
Complete	Yes, with finite branching and a finite-depth solution
Cost-optimal	Only when every action has the same cost
Tree-like time	$O(b^d)$ with the early goal test
Tree-like space	$O(b^d)$

A small depth can still be large

If $b = 10$ and $d = 6$,

$$1 + 10 + \dots + 10^6 = 1,111,111.$$

Most generated nodes sit at the deepest frontier level.

Breadth-First Tradeoffs

b : maximum branching factor d : shallowest solution depth

Property	Breadth-first search
Complete	Yes, with finite branching and a finite-depth solution
Cost-optimal	Only when every action has the same cost
Tree-like time	$O(b^d)$ with the early goal test
Tree-like space	$O(b^d)$

A small depth can still be large

If $b = 10$ and $d = 6$,

$$1 + 10 + \dots + 10^6 = 1,111,111.$$

Most generated nodes sit at the deepest frontier level.

Strength

Reliable
shallow solutions

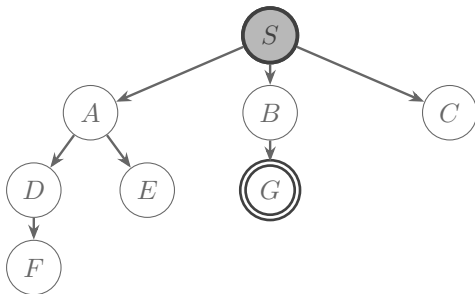
Cost

Exponential
frontier memory

Warning

Shallowest need
not be cheapest

Depth-First Ordering



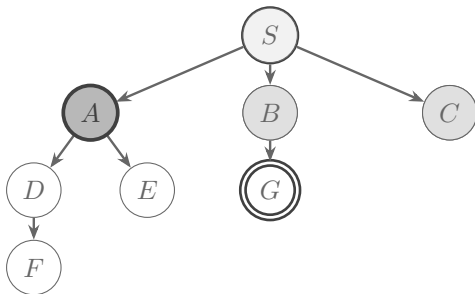
Rule

Select the deepest node. A LIFO stack makes the newest path continue first.

Stack, top first: S

Successors are listed left to right and pushed in reverse order.

Depth-First Ordering



Rule

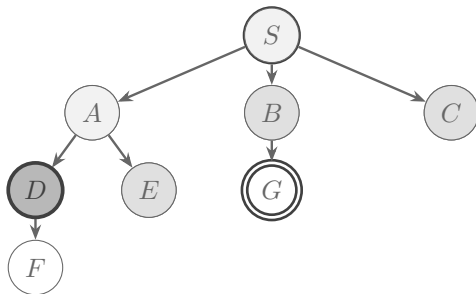
Select the deepest node. A LIFO stack makes the newest path continue first.

After expanding S :



Successors are listed left to right and pushed in reverse order.

Depth-First Ordering



Rule

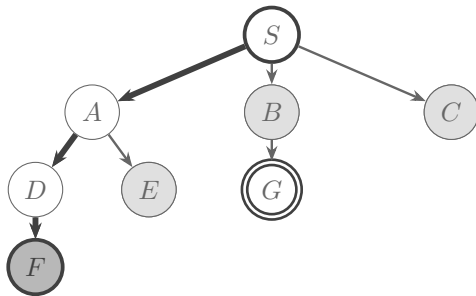
Select the deepest node. A LIFO stack makes the newest path continue first.

After expanding A:



Successors are listed left to right and pushed in reverse order.

Depth-First Ordering



Rule

Select the deepest node. A LIFO stack makes the newest path continue first.

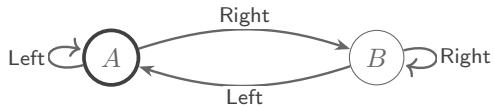
Continue the branch:

F E B C

Successors are listed left to right and pushed in reverse order.

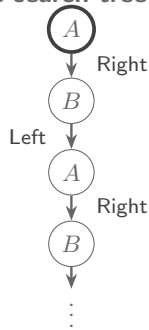
Cycles and Repetition

Finite vacuum-world fragment



Two locations, four directed transitions

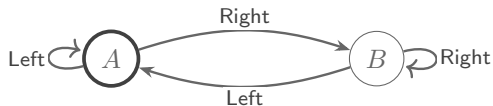
Infinite search-tree branch



Repeated states become new nodes

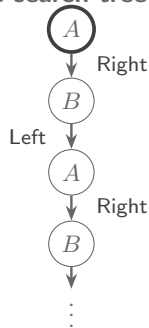
Cycles and Repetition

Finite vacuum-world fragment



Two locations, four directed transitions

Infinite search-tree branch

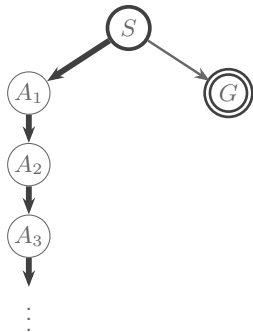


Repeated states become new nodes

Check	What it prevents
Current-path cycle check	Returning to an ancestor on this branch
Global reached set	Cycles and reconverging duplicate paths

Depth-First Tradeoffs

b : maximum branching factor m : maximum search depth
 $|V|$: number of represented states



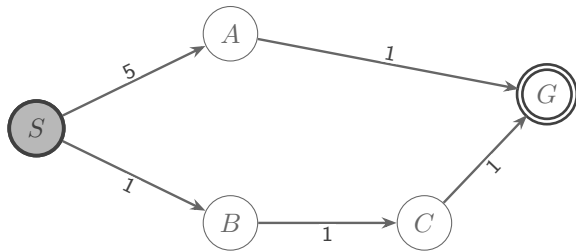
Left-first DFS can follow an infinite branch while a depth-1 goal waits.

Property	Depth-first search
Complete	Finite graph with reached: yes; not in general
Cost-optimal	No
Tree-like time	$O(b^m)$
Tree-like space	$O(bm)$

Tradeoff

Small frontier memory can come with long detours, order sensitivity, and no solution guarantee in infinite-depth spaces. A global reached set adds $O(|V|)$ storage in a finite graph.

Uniform-Cost Ordering

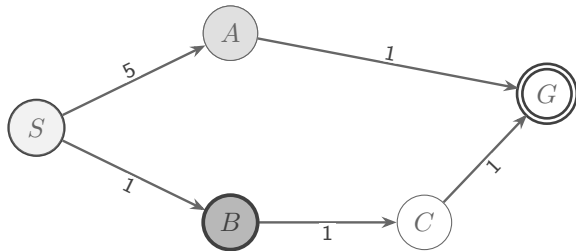


Rule

Select the frontier node with minimum accumulated path cost $g(n)$.

Priority frontier: $S : 0$

Uniform-Cost Ordering



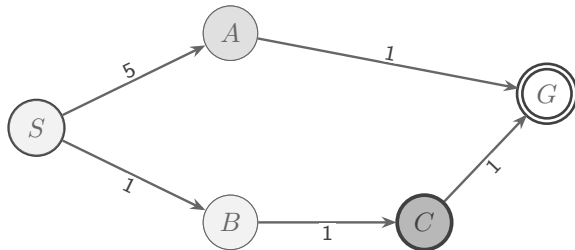
Rule

Select the frontier node with minimum accumulated path cost $g(n)$.

After S :

$B : 1$ $A : 5$

Uniform-Cost Ordering



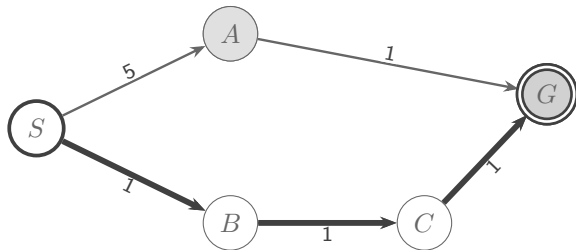
Rule

Select the frontier node with minimum accumulated path cost $g(n)$.

After B:

C : 2 A : 5

Uniform-Cost Ordering



Rule

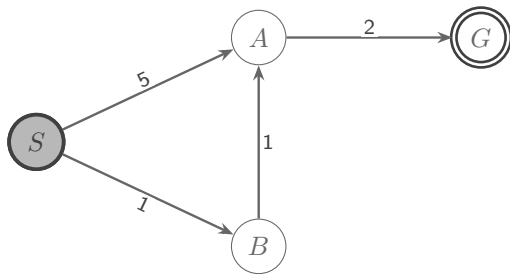
Select the frontier node with minimum accumulated path cost $g(n)$.

After C:

$G : 3$ $A : 5$

Select $G : 3$ and return.

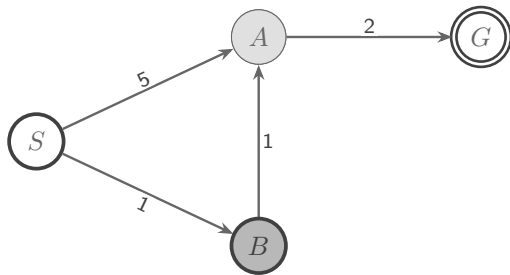
Frontier Replacement



Start

Frontier: S : 0

Frontier Replacement

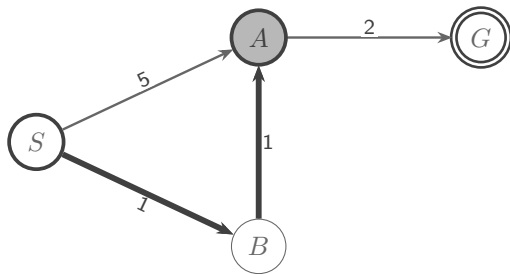


Direct route to A

Frontier: $B : 1$ $A : 5$

A Boolean visited flag is insufficient for UCS: reached must retain the best known cost and parent.

Frontier Replacement



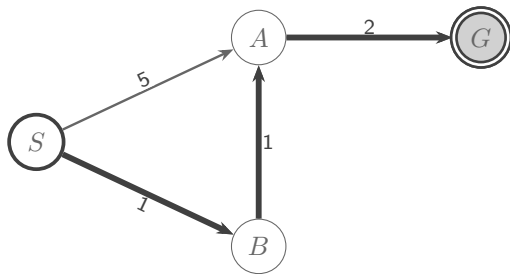
Cheaper route discovered

$S \rightarrow B \rightarrow A$ costs 2.

Replace $A : 5$ with $A : 2$ and reparent A through B .

A Boolean visited flag is insufficient for UCS: reached must retain the best known cost and parent.

Frontier Replacement



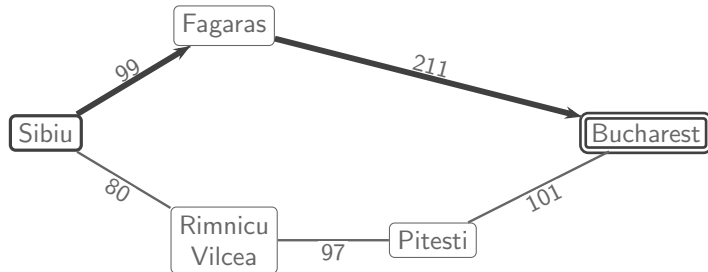
Recovered solution

$S \rightarrow B \rightarrow A \rightarrow G$

$$g = 1 + 1 + 2 = 4$$

A Boolean visited flag is insufficient for UCS: reached must retain the best known cost and parent.

Goal-Test Timing



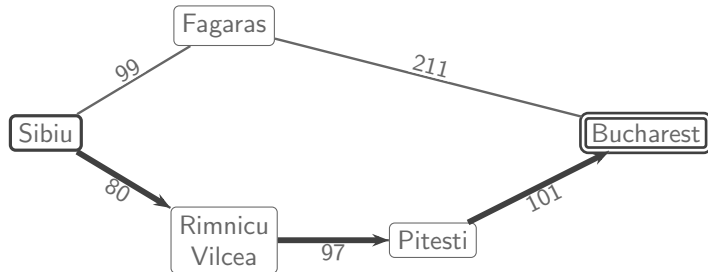
First goal generated

Sibiu–Fagaras–Bucharest

$$g = 99 + 211 = 310$$

Should UCS return now?

Goal-Test Timing



Cheaper goal found later
Sibiu–Rimnicu–Pitesti–Bucharest

$$g = 80 + 97 + 101 = 278$$

No early return.

BFS may test a newly generated goal because depth cannot later improve. UCS returns a goal only when it is selected as the minimum- g frontier node.

Uniform-Cost Tradeoffs

b : maximum branching factor C^* : optimal solution cost
 ϵ : minimum positive step cost $g(n)$: path cost to node n

Property	Uniform-cost search
Complete	Finite branching; every action costs at least $\epsilon > 0$
Cost-optimal	Yes, with those assumptions
Tree-like time	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
Tree-like space	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$

Expanding cost contours

UCS selects nodes in nondecreasing g .

Many low-cost steps can create a large frontier before the optimal goal is selected.

Uniform-Cost Tradeoffs

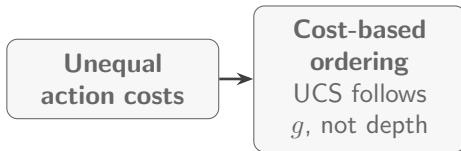
b : maximum branching factor C^* : optimal solution cost
 ϵ : minimum positive step cost $g(n)$: path cost to node n

Property	Uniform-cost search
Complete	Finite branching; every action costs at least $\epsilon > 0$
Cost-optimal	Yes, with those assumptions
Tree-like time	$O(b^{1+\lceil C^*/\epsilon \rceil})$
Tree-like space	$O(b^{1+\lceil C^*/\epsilon \rceil})$

Expanding cost contours

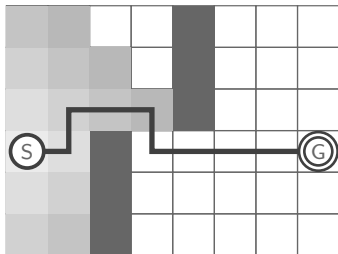
UCS selects nodes in nondecreasing g .

Many low-cost steps can create a large frontier before the optimal goal is selected.



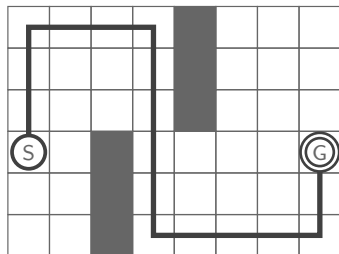
Grid Search Patterns

BFS: expanding wavefront



Darker cells lie in later BFS layers (shown through depth 4).

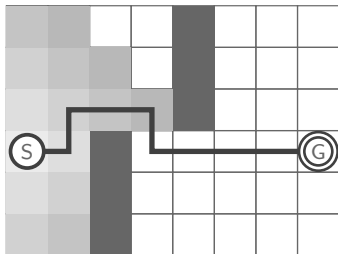
DFS: one branch at a time



Commits to the successor order, even on a long detour.

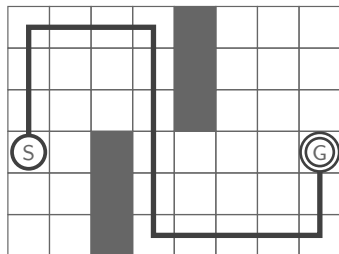
Grid Search Patterns

BFS: expanding wavefront



Darker cells lie in later BFS layers (shown through depth 4).

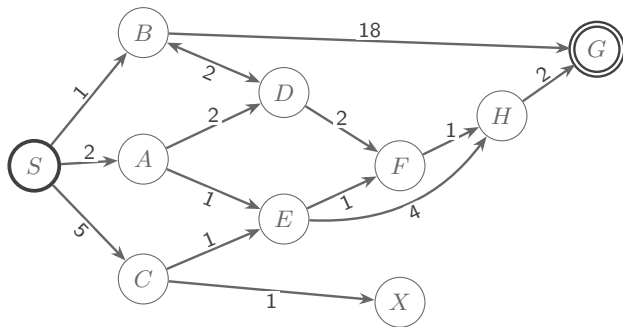
DFS: one branch at a time



Commits to the successor order, even on a long detour.

With unit move costs, UCS and BFS induce the same depth ordering; tie-breaking may change which equal-cost node appears first.

Search Strategy Challenge



Successor order

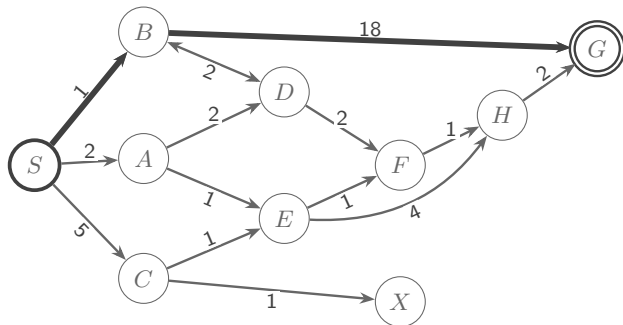
<i>S</i>	<i>A</i> (2), <i>B</i> (1), <i>C</i> (5)
<i>A</i>	<i>D</i> (2), <i>E</i> (1)
<i>B</i>	<i>D</i> (2), <i>G</i> (18)
<i>C</i>	<i>X</i> (1), <i>E</i> (1)
<i>D</i>	<i>F</i> (2), <i>B</i> (2)
<i>E</i>	<i>F</i> (1), <i>H</i> (4)
<i>F</i>	<i>H</i> (1)
<i>H</i>	<i>G</i> (2)

Trace conventions

BFS tests goals on generation. DFS and UCS test on selection. DFS visits the first listed successor first. UCS breaks *g* ties by earlier insertion.

**For each strategy: trace the frontier to termination;
record the returned path, depth, cost, and peak frontier.**

Breadth-First Challenge



Expand	FIFO frontier afterward
<i>S</i>	[<i>A</i> , <i>B</i> , <i>C</i>]
<i>A</i>	[<i>B</i> , <i>C</i> , <i>D</i> , <i>E</i>]
<i>B</i>	<i>D</i> ignored; then <i>G</i> generated

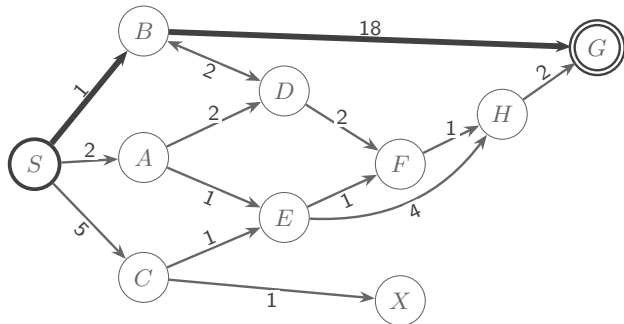
Return on generation

$$S \rightarrow B \rightarrow G$$

$$\text{depth} = 2, \quad g = 1 + 18 = 19$$

Peak frontier size: 4.

Breadth-First Challenge



Expand	FIFO frontier afterward
<i>S</i>	[<i>A, B, C</i>]
<i>A</i>	[<i>B, C, D, E</i>]
<i>B</i>	<i>D</i> ignored; then <i>G</i> generated

Return on generation

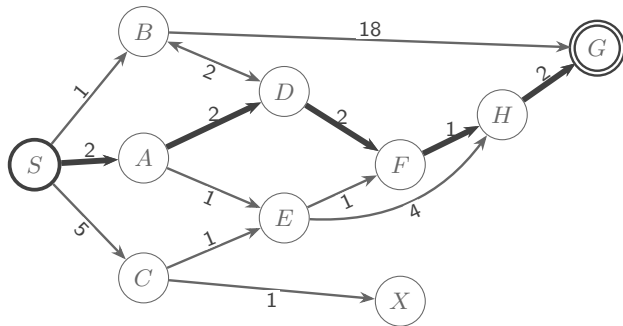
$$S \rightarrow B \rightarrow G$$

depth = 2, $g = 1 + 18 = 19$

Peak frontier size: 4.

When *B* proposes *D* with cost 3, BFS ignores it:
D was already reached from *A* at the same depth, even though that path cost 4.

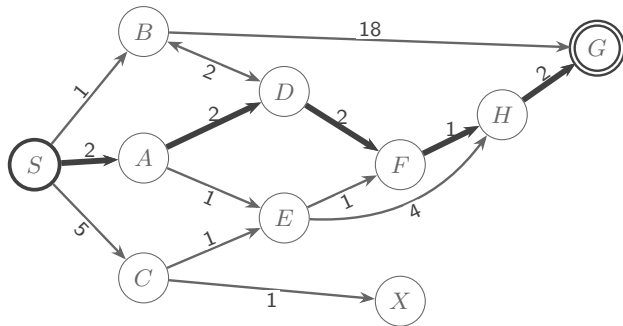
Depth-First Challenge



Select	Stack afterward, top first
<i>S</i>	[<i>A</i> , <i>B</i> , <i>C</i>]
<i>A</i>	[<i>D</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>D</i>	[<i>F</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>F</i>	[<i>H</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>H</i>	[<i>G</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>G</i>	return

Peak frontier size: 4.

Depth-First Challenge



Select	Stack afterward, top first
<i>S</i>	[<i>A</i> , <i>B</i> , <i>C</i>]
<i>A</i>	[<i>D</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>D</i>	[<i>F</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>F</i>	[<i>H</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>H</i>	[<i>G</i> , <i>E</i> , <i>B</i> , <i>C</i>]
<i>G</i>	return

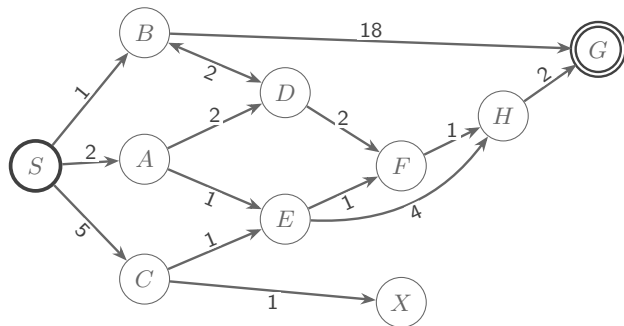
Peak frontier size: **4**.

Returned path: $S \rightarrow A \rightarrow D \rightarrow F \rightarrow H \rightarrow G$

depth 5, cost $2 + 2 + 2 + 1 + 2 = \mathbf{9}$.

At *D*, reached discards *B* and cuts the $B \leftrightarrow D$ cycle.

Uniform-Cost Challenge I



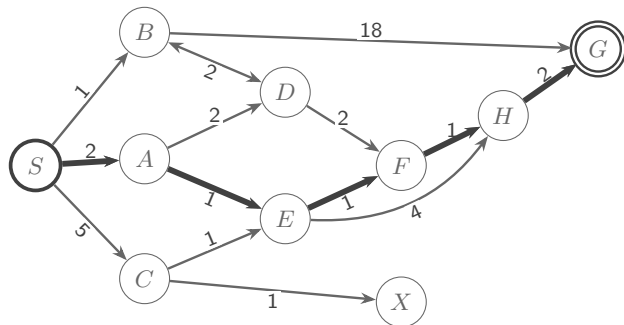
Select	Priority frontier afterward
$S : 0$	$[B : 1, A : 2, C : 5]$
$B : 1$	$[A : 2, D : 3, C : 5, G : 19]$
$A : 2$	$[D : 3, E : 3, C : 5, G : 19]$
$D : 3$	$[E : 3, C : 5, F : 5, G : 19]$
$E : 3$	$[F : 4, C : 5, H : 7, G : 19]$

First replacement

E finds F with cost 4, replacing the frontier entry $F : 5$ generated from D .

Equal- g ties use earlier insertion: $D : 3$ is selected before $E : 3$.

Uniform-Cost Challenge II



Select	Priority frontier afterward
<i>F</i> : 4	[<i>C</i> : 5, <i>H</i> : 5, <i>G</i> : 19]
<i>C</i> : 5	[<i>H</i> : 5, <i>X</i> : 6, <i>G</i> : 19]
<i>H</i> : 5	[<i>X</i> : 6, <i>G</i> : 7]
<i>X</i> : 6	[<i>G</i> : 7]
<i>G</i> : 7	return

Further replacements

F improves *H* : 7 to *H* : 5.

H improves *G* : 19 to *G* : 7.

Returned path: $S \rightarrow A \rightarrow E \rightarrow F \rightarrow H \rightarrow G$

depth 5, cost **7**.

Challenge Debrief

Strategy	Returned path	Depth	Cost	Peak frontier
BFS	$S \rightarrow B \rightarrow G$	2	19	4
DFS	$S \rightarrow A \rightarrow D \rightarrow F \rightarrow H \rightarrow G$	5	9	4
UCS	$S \rightarrow A \rightarrow E \rightarrow F \rightarrow H \rightarrow G$	5	7	4

Challenge Debrief

Strategy	Returned path	Depth	Cost	Peak frontier
BFS	$S \rightarrow B \rightarrow G$	2	19	4
DFS	$S \rightarrow A \rightarrow D \rightarrow F \rightarrow H \rightarrow G$	5	9	4
UCS	$S \rightarrow A \rightarrow E \rightarrow F \rightarrow H \rightarrow G$	5	7	4

BFS

The shallow goal wins, even though its edge cost is large.

DFS

The first-listed deep branch determines the returned route.

UCS

Cheaper-path replacement allows the lowest-cost route to emerge.

The equal peak frontier is accidental; the strategies still select and retain different nodes.

Strategy Guarantees

	BFS	DFS	UCS
Selection	Shallowest node	Deepest node	Lowest $g(n)$
Frontier	FIFO queue	LIFO stack	Min-priority queue
Goal test	On generation	On selection	On selection
Complete	Finite b ; finite goal depth d	Finite graph with reached; not in general	Finite b ; costs $\geq \epsilon > 0$
Cost-optimal	Equal action costs only	No	Yes, if costs $\geq \epsilon > 0$

The decisive question

Do we care first about number of actions, frontier memory, or total path cost?

Search Complexity

	BFS	DFS	UCS
Tree-like time	$O(b^d)$	$O(b^m)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
Tree-like space	$O(b^d)$	$O(bm)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
Global reached	$O(V)$	$O(V)$	$O(V)$ (best costs)
Main risk	Frontier memory	Detours; infinite branches	Many cheap paths

How to read the bounds

The exponential bounds describe tree-like growth before duplicate merging. In an explicit finite graph, a global reached or best-cost table adds $O(|V|)$ storage and exploration is limited by the represented vertices and edges.

d : shallowest goal depth m : maximum depth C^* : optimal cost step cost $\geq \epsilon > 0$

Search Takeaways

Breadth-first search
shallowest first

FIFO

Reliable for minimum-action
solutions; memory intensive.

Depth-first search
deepest first

LIFO

Small frontier; order sensitive
and not generally optimal.

Uniform-cost search
lowest g first

$$f(n) = g(n)$$

Cost-optimal under
positive-cost assumptions;
updates matter.

Search Takeaways

Breadth-first search
shallowest first

FIFO

Reliable for minimum-action solutions; memory intensive.

Depth-first search
deepest first

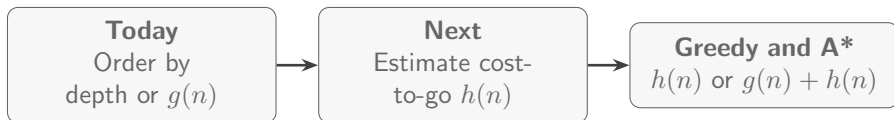
LIFO

Small frontier; order sensitive and not generally optimal.

Uniform-cost search
lowest g first

$$f(n) = g(n)$$

Cost-optimal under positive-cost assumptions; updates matter.



Thank You

Any questions?

References

S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. URL <https://aima.cs.berkeley.edu/>.